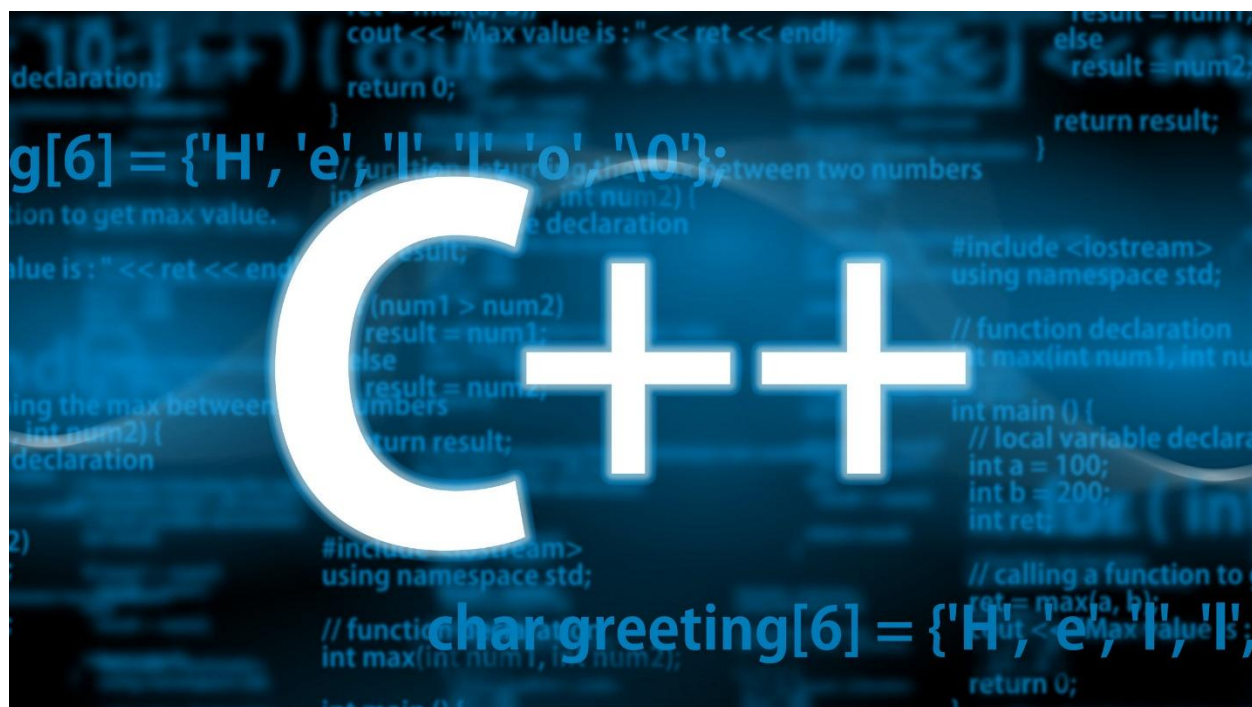


C++ در قرن بیست و یکم^۱

معرفی مکانیزم های کلیدی طراحی شده در C++ امروزین با نگهداشت سازگاری در طول چند دهه



نویسنده: بیارنه استراستروپ

برگردان: سعید امراللهی بیوکی

کارشناس ارشد مهندسی نرم افزار

^۱ این نوشتار ترجمه یا برگردان مقاله ی زیر است:

Bjarne Stroustrup. 21st Century C++. Blog@CACM. February 2025. <https://cacm.acm.org/blogcacm/21st-century-c/>

همچنین می توانید نسخه پی دی اف این مقاله را از پیوند زیر بارگیری کنید:

<https://www.stroustrup.com/21st-Century-C++.pdf>

چکیده

بیش از ۴۵ سال از شکل‌گیری زبان C++ می‌گذرد. این زبان به گونه‌ای تکامل یافت تا با چالش‌های زمان خود مواجه شود، ولی بسیاری از برنامه‌سازان، C++ را آنگونه بکار می‌گیرند که گویی هنوز در هزاره‌ی دوم هستیم! این شکل استفاده از زبان، از منظر سهولت در بیان ایده‌ها، کارایی، اطمینان‌پذیری و نگهداشت‌پذیری بهینه نیست. در اینجا می‌خواهم، چند مفهوم کلیدی را عرضه کنم که با آنها می‌توان کدهایی کارآمد، ایمن‌گونه^۲ و منعطف نوشت. این مفاهیم عبارتند از: مدیریت منابع، مدیریت زیست، مهار خطا، واحد‌مند بودن و برنامه‌سازی هرویک. در پایان تکنیک‌هایی عرضه خواهد شد تا به کمک آنها بتوانیم کدهای امروزی را از کدهای منسوخ، ناایمن و با نگهداری دشوار متمایز کنیم. این ابزار عبارتند از: رهنمون‌ها و نمایه‌ها.

این مقاله برای کسانی که دانش خوبی از برنامه‌سازی مدرن دارند قابل فهم است ولی برای افرادی با دانش اولیه‌ای در C++ هدف‌گذاری شده است. این مقاله یک آموختار نیست بلکه هدف آن تقریر کمال مطلوب، اصول و تکنیک‌ها همراه با قطعه کدهای کوتاه می‌باشد.

۱. مقدمه

C++ زبانی است با تاریخچه‌ای طولانی. برنامه‌سازان، معلمان و دانشجویان دهه‌ها پیشرفت را نادیده می‌گیرند و C++ را زبانی از هزاره‌ی دوم توصیف می‌کنند، روزگاری که باید تلفنها به پرز دیوار زده می‌شد و اکثر برنامه‌ها کوتاه، سطح پایین و کند بودند.

اگر سیستم عامل کامپیوتر شما در چند دهه‌ی گذشته سازگاری خود را حفظ کرده باشد، می‌توانید برنامه‌های C++ ای که در سال ۱۹۸۵ میلادی نوشته شده را همین حالا بر روی کامپیوترهای نوین اجرا کنید. استواری - به معنای سازگاری با ویرایش‌های قدیمی تر زبان - بی‌اندازه مهم است، به‌ویژه برای سازمان‌هایی که سامانه‌های نرم‌افزاری را برای دهه‌ها {استفاده و} نگهداری می‌کنند. با این حال، تقریباً در همه حالات، با C++ امروزی [30] می‌توان ایده‌های موجود در کدهای قدیمی را بیان کرد: ساده تر و با تضمین امنیت گونه‌ی بسیار بهتر، با اجرای سریع‌تر و البته مصرف حافظه کمتر.

این مقاله مکانیزم‌های کلیدی C++ امروزی را برای برآورده ساختن این قابلیت‌ها عرضه می‌کند. در پایان (بخش ۶)، به توصیف تکنیک‌هایی می‌پردازیم که برنامه‌ساز را مجبور می‌سازد تا این شیوه‌ی نوین را استفاده کند.

برنامه‌ی ساده‌ی زیر را در نظر بگیرید. این برنامه خطها را از ورودی می‌خواند و با حذف تکراری‌ها، خطوط یکتا را در خروجی می‌نویسد^۴:

```
import std; // make all of the standard library available
using namespace std;
int main() // print unique lines from input
{
    unordered_map<string,int> m; // hash table
    for (string line; getline(cin,line); )
        if (m[line]++ == 0)
            cout << line << '\n';
}
```

^۲ Type-safe

^۳ در انتخاب برابری‌های فنی به ترتیب از واژه نامه انجمن انفورماتیک ایران، واژه‌های مصوب فرهنگستان، وبگاه واژه یاب (www.vajehyab.com)، واژه‌نامه مهندسی داده‌ها از آقای روحانی رانکوهی و پیشنهادهای گروه تلگرامی «برابر یابی برای واژه‌های تخصصی» استفاده شده است.

^۴ تمامی برنامه‌ها با کامپایلر GCC 15.2.0 از GNU در محیط لینوکس کامپایل و اجرا شده‌اند.

افراد خبره {و آشنا با یونیکس}، می‌توانند تشخیص دهند که این کد همان کاری را انجام می‌دهد که کد (!a\$0)++ در زبان AWK، انجام می‌دهد. این برنامه از unordered_map که ویرایش جدول درهم سازی^۵، در کتابخانه‌ی استاندارد C++ می‌باشد، بهره می‌برد. با استفاده از این جدول، خطهای یکتا را نگه می‌داریم و فقط در صورتیکه یک خط برای اولین بار دیده شد، در خروجی نمایش داده خواهد شد.

حلقه‌ی for، برای محدود کردن حوزه یا قلمرو^۶ متغیر حلقه (line) استفاده شده است { تا خارج از حلقه قابل دسترسی نباشد }. در مقایسه با سبکهای قدیمی‌تر C++، عدم استفاده‌ی صریح از اقلام زیر قابل ملاحظه است:

- تخصیص و آزادسازی حافظه
 - اندازه‌ها اعم از طول جدول درهم ساز، رشته نویسه، ورودی و خروجی
 - مدیریت خطا^۷
 - تبدیل گونه (قالب دهی)^۸
 - نشانه‌روها
 - عملگر اندیس گذاری یا انتخاب {منظور [] است که می‌تواند ناایمن باشد}
 - استفاده از پیش پردازنده (همانطور که می‌بینید خبری از اعلانهای #include نیست)
- همچنان، این برنامه در قیاس با سبک های قدیمی تر، بسیار کارا است، کاراتر از اکثر کدهایی که برنامه سازان بخواهند در مدت زمانی منطقی خودشان بنویسند. اگر به کدی کارآمدتر نیاز داشته باشیم باید برنامه را تنظیم کنیم. یک جنبه‌ی مهم C++ این است که کدهای با رابط مناسب را می‌توان بر اساس نیازهای مشخص و حتی سخت افزار اختصاصی تنظیم کرد این کار بدون اختلال در دیگر کدها و معمولاً بدون تغییر در خود کامپایلر قابل اعمال است.
- بدیل برنامه‌ی قبل را در نظر بگیرید. این برنامه خطوط یکتا را برای استفاده های آتی ذخیره می‌کند:

```
import std; // make all of the standard library available
using namespace std;
vector<string> collect_lines(istream& is) // collect unique lines from input
{
    unordered_set<string> s; // hash table
    for (string line; getline(is,line); )
        s.insert(line);
    return vector{from_range, s}; // copy set elements into a vector
}
auto lines = collect_lines(cin);
```

- در کد فوق چند نکته قابل توجه است:
- از آنجا که دیگر به تعداد خطوط نیازی نیست، بجای نگاشت از مجموعه استفاده کرده‌ام.
 - تابع collect_lines، یک vector بر می‌گرداند، نه یک set، زیرا بردار پر استفاده ترین ظرف { و همچنین کارآمدتر { است.
 - نیازی نبود گونه اعضای بردار را مشخص کنم، زیرا کامپایلر آنرا از روی گونه‌ی عضو مجموعه می‌تواند استنتاج کند.

^۵ Hash table

^۶ Scope

^۷ Error-handling

^۸ Cast

- از نشانوند **from_range** استفاده کرده ام تا به کامپایلر و کاربر انسانی بگویم که `{}` از میان روشهای مختلف آغازین دهی بردار `{}` از یک محدوده^۹ برای پر کردن آن استفاده شده. من خودم ترجیح می‌دهم از شکل کمینه **vector{m}** استفاده کنم اما کمیته استاندارد سازی استفاده از **from_range** را ملزم کرد، چون به خوانایی کد کمک زیادی می‌کند.

برنامه سازان مجرب، متوجه هستند که **collect_lines()** `{}` هنگام فراخوانی `{}` نویسه ها را کپی می‌کند. این می‌تواند مشکل کارآمدی به همراه داشته باشد، در بخش ۳.۲، برنامه را تنظیم خواهیم کرد تا از کپی داده ها جلوگیری کنیم. اهمیت این مثال‌های ساده در چیست؟ نمایش C++ امروزین قبل از آنکه وارد جزئیات فنی شویم و امیدوار باشیم که تا حدی پایه‌های سوء تفاهم های غیر صمیمانه و چند دهه‌ای افراد را لرزان کرده باشیم.

۲. کمال گرایی به سبک C++

هدف گذاری من برای یک C++ ایده آل چنین بوده است:

- توصیف مستقیم ایده ها در برنامه
- ایمنی ایستای گونه `{}` منظور ایمنی گونه در زمان کامپایل است. `{}`
- ایمنی منابع (یا عدم نشت منابع)
- دسترسی مستقیم به سخت‌افزار
- کارآمدی (یا کارایی)
- برای آنچه که استفاده نمی‌شود، نگران پرداخت هزینه نباشید (قانون سربار^{۱۰} صفر)
- گسترش پذیری مقرون به صرفه (یا انتزاع های با سربار صفر)
- پشتیبانی از همروندی (از طریق کتابخانه ها)
- نگهداشت پذیری (یا کدهای قابل فهم)
- ناپستگی به سکوی نرم‌افزار/سخت‌افزار (یا حمل پذیری)
- استواری (یا سازگاری)

این خواسته‌ها، نسبت به روزهای نخست تغییری نکرده است [24]، اما قرار بوده که C++ تکامل یابد و C++ امروزین می‌تواند این جنبه‌ها را نسبت به ویرایش های قدیمی تر بسیار بهتر ارائه دهد. کدهایی که دربردارنده‌ی این کمال‌گرایی هستند صرفاً با بکارستن جدیدترین جنبه های زبان به این سطح مطلوب نرسیده‌اند. برخی از جنبه‌ها و تکنیک‌های کلیدی قدیمی هستند از جمله:

- رده های مجهز به ساختگر و ویرانگر
- استثناءها و مدیریت آنها
- الگوها
- بردارهای استاندارد
- ...

ویژگیهای دیگر جدیدتر هستند:

^۸Range

^۹Overhead

- ماجول‌ها (بخش ۴)
- مفهوم‌ها (برای بیان رابط‌های هرویک، بخش ۵.۲)
- عبارت‌های لاند (برای تولید شیء تابع‌ها، بخش ۵.۱)
- محدوده‌ها
- عبارت‌های ثابت و ارزیابی‌های ثابت (برای محاسبات در زمان کامپایل، بخش ۵.۲)
- پشتیبانی از همروندی و الگوریتم‌های موازی
- همروال‌ها (اگرچه دهه‌ها غایب بوده ولی بخشی اساسی در نسخه اولیه C++ بوده است)
- نشانه‌رو‌های هوشمند مانند `unique_ptr` و `shared_ptr`
- ...

آنچه مهم است این است که قابلیت‌ها و جنبه‌های زبان و کتابخانه را به عنوان یک مجموعه منسجم و به هم پیوسته برای حل مسائل ببینیم.

آنچه که به یک زبان برنامه‌سازی ارزش می‌دهد، گستره و کیفیت کاربردهای آن است. در مورد C++، شواهد از گستره حیرت‌آور، کاربردهای آن حکایت می‌کند: نرم‌افزارهای بنیادین (اعم از سیستم‌های عامل و بانک‌های اطلاعاتی)، گرافیک کامپیوتری، نرم‌افزارهای علمی، صنعت سینما، بازی‌ها، خودروها، پیاده‌سازی زبانهای برنامه‌سازی (که فقط شامل پیاده‌سازی C++ نمی‌شود)، نرم‌افزارهای کنترل پرواز، موتورهای جستجو، مرورگرها، طراحی و ساخت نیمه‌هادی‌ها و کاوش‌های فضایی، نرم‌افزارهای مالی، هوش مصنوعی و بسیاری دیگری از حوزه‌ها. با وجود بیلیون‌ها خط برنامه C++، ما نمی‌توانیم ناسازگاری‌ها را بهبود دهیم ولی می‌توان شیوه استفاده از C++ را تغییر داد.

در ادامه، تمرکز من بر روی موضوعات زیر است:

- مدیریت منابع (شامل کنترل طول عمر و مدیریت خطا)
- ماجول‌ها (شامل حذف پیش پردازنده)
- برنامه‌سازی هرویک (شامل مفهوم‌ها یا ادراک‌ها)
- رهنمون‌ها و الزام‌آورها (برای ضمانت "هزاره‌ی سومی بودن C++ ای" که به کار می‌بریم)

طبیعتاً، این همه‌ی آن چیزی نیست که C++ عرضه می‌کند و کدهای بسیار خوبی به شیوه‌هایی نوشته می‌شوند که از سبک‌هایی غیر از موارد فوق استفاده می‌کنند. به عنوان مثال، من برنامه‌سازی شیء‌گرا را در فهرست قرار ندادم، زیرا عده زیادی از برنامه‌سازان می‌دانند که چگونه با این الگوی برنامه‌سازی کار کنند. به علاوه، کدهای بسیار بس کارآمد^{۱۱} و برنامه‌هایی که مستقیماً با سخت‌افزار کار می‌کنند به مراقبت ویژه و تکنیک‌هایی متفاوت نیاز دارند. همچنین پشتیبانی مبسوط و گسترده‌ی همروندی در C++ به مقاله‌ای مجزا نیاز دارد. به هر روی، کلید اکثر کدهای با کیفیت، رابط‌های ایمن‌گونه است که حاوی اطلاعات کافی برای بهینه‌سازی و بررسی خصوصیات شیء‌ها در زمان اجرا می‌باشند، آنچه که در زمان کامپایل تضمین شده نیست.

۳. مدیریت منابع

منبع هر چیزی است که ما باید در اختیار بگیریم و دیرتر به شکل صریح یا ضمنی آزادش کنیم یا پس بدهیم. حافظه، قفل، مهار فایل^{۱۲}، سوکت، مهار ریسه^{۱۳}، تراکش و سایه‌دار^{۱۴} نمونه‌هایی از منبع در برنامه‌سازی هستند. برای جلوگیری از نشت

^{۱۱} High-performance

^{۱۲} File handle

^{۱۳} Thread handle

^{۱۴} Shader

منابع، باید از آزادسازی آن به شکل دستی (یا صریح) پرهیز کرد. شوربختانه، افراد - حتی برنامه سازان خوب - در پس دادن آنچه که قرض گرفته اند، خوش حافظه نیستند!

تکنیک بنیادین C++ برای مدیریت منابع، ریشه در این نکته دارد که مهار یک منبع به محض پایان قلمرو، آزاد شود. برای نیل به اطمینان پذیری، ما نمی‌توانیم بر استفاده‌ی صریح `delete`، `free()`، `unlock()` و غیره در کد اصلی تکیه کنیم. این عملیات به کد مهار منابع { که در کد اصلی صرفاً فراخوانی می‌شود } تعلق دارد. برای روشن شدن موضوع، قطعه کد زیر را در نظر بگیرید:

```
template<typename T>
class Vector { // vector of elements of type T
public:
    Vector(initializer_list<T>); // constructor: acquire memory; initialize
                                // elements
    ~Vector(); // destructor: destroy elements; release memory
    // ...
private:
    T* elem; // pointer to elements
    int sz;  // number of elements
};
```

در اینجا **Vector**، یک مهار منبع است. **Vector** سطح انتزاع را از یک زوج نشانه رو و تعداد المانهایی که به آن نشانه رفته است به یک گونه‌ی نامدار و مناسب با آغازین‌دهی تضمین‌شده‌ی حافظه (به کمک ساختگر) و پاک کردن و برگرداندن آن (با کمک ویرانگر) بالا برده است.

vector استاندارد که این **Vector** قرار است آنرا تصویر کند عملیات مقایسه، گمارش، شیوه‌های بیشتر آغازین‌دهی، تغییر اندازه، پیمایش المانها و بسیاری امکانات دیگر را پشتیبانی می‌کند. این باعث می‌شود که از منظر فنی زبان (مثلاً نامگذاری، قلمرو، نمادگذاری و ...)، برنامه ساز با داده گونه‌ای کار کند -منظور **vector** است- که رفتاری شبیه به داده گونه‌های توکار - مانند `int` - دارد اگرچه یک مهار منبع در کتابخانه است و معنایی بسیار متفاوت دارد. یک نمونه از بکار بستن **Vector** اینگونه است:

```
void fct()
{
    Vector<double> constants { 1, 1.618, 3.14, 2.99e8 };
    Vector<string> designers { "Strachey", "Richards", "Ritchie" };
    // ...
    Vector<pair<string,jthread>> vp { { "producer", prod },
                                     { "consumer", cons } };
}
```

در کد فوق، **constants**، با چهار مقدار ریاضی و فیزیکی آغازین‌دهی شده است. **designers** با نام سه طراح شناخته شده زبانهای برنامه سازی و **vp** با یک زوج از تولید کننده و مصرف کننده^{۱۵}. همه این بردارها با ساختگرهای مناسب آغازین‌دهی می‌شوند و در پایان یا خروج قلمرو - هنگامی که آکولاد بسته توسط کامپایلر یا پردازنده مشاهده می‌شود - ویرانگرهای درخور فراخوانی می‌شوند. آغازین‌دهی و آزادسازی، عملیاتی خود بازگشتی هستند. مثلاً، ساخت و تخریب بردار **vp**، عملیاتی نابدیهی است، زیرا

ساختمان داده ای خود بازگشتی است از رده های **string**، **pair**، **Vector** (مهار رشته نویسه ها) و **jthread** (مهار ریشه در سطح سیستم عامل). همه این عملیات به شکل ضمنی انجام می شود.

استفاده از زوج ساختگر-ویرانگر (که اغلب با عنوان "در اختیار گرفتن منبع به معنای آغازین دهی است"^{۱۶} از آن یاد می شود)، فقط آزادسازی منابع را ضمانت نمی کند، بلکه از انباشت و احتباس آنها نیز جلوگیری می کند بنابراین به جهت کارایی مزیت قابل توجهی در قیاس با تکنیکهای دیگری مانند مدیریت حافظه برپایه ی زباله روب^{۱۷} عرضه خواهد کرد.

۱.۳. کنترل طول عمر

کنترل کردن طول عمر یا زمان زیست شیء دربرگیرنده ی یک منبع، ضروری است تا مدیریت منابع ساده و کارا باشد. برای این منظور، ++C ۴ ابزار کنترلی دارد که در قالب عملیات یک رده تعریف می شود (X نام رده است):

- ساختن: این عمل، قبل از اولین استفاده از شیء فراخوانی می شود تا اگر رده ناوردایی^{۱۸} دارد، آنرا پی ریزی کند. نام این عمل ساختگر^{۱۹} است با امضای **X(optional_arguments)**.
- تخریب: این عمل، پس از آخرین استفاده از شیء فراخوانی می شود تا همه ی منابع در اختیار را آزاد سازد. نام این عمل ویرانگر^{۲۰} است با امضای **~X()**.
- کپی کردن: این عمل، از یک شیء، یک کپی جدید و کاملاً یکسان می سازد (برای گونه های با قاعده^{۲۱}). پس از اجرای **a = b** خواهیم داشت: **a == b**. این عمل خود از دو تابع عضو مجزا تشکیل شده است: ساختن با کپی^{۲۲} با امضای **X(const X&)** و عملگر گماردن^{۲۳} با کپی^{۲۴} با امضای **X::operator=(const X&)**.
- جابه جا کردن^{۲۵}: این عمل منابع را از یک شیء به شیء ای دیگر که اغلب بین قلمروها صورت می گیرد، جابجا می کند. این عمل خود از دو تابع عضو مجزا تشکیل شده است: ساختن با جابجایی^{۲۶} **X(X&&)** و عملگر گماردن با جابجایی^{۲۷} **X::operator(X&&)**.

برای مثال، می توان توصیف رده **Vector** را به شکل زیر ادامه داد:

```
template<typename T>
class Vector { // vector of elements of type T
public:
    Vector(); // default constructor: make an empty vector
    Vector(initializer_list<T>); // constructor: acquire memory;
                                   // initialize elements
```

^{۱۶} Resource Acquisition Is Initialization- RAII در این مقاله به اختصار از عبارت "تکنیک اکتساب و آغازین دهی" نیز استفاده شده است.

^{۱۷} Garbage collector

^{۱۸} Invariant

^{۱۹} Constructor

^{۲۰} Destructor

^{۲۱} Regular types

^{۲۲} Copy constructor

^{۲۳} Assignment

^{۲۴} Copy assignment operator

^{۲۵} Move

^{۲۶} Move constructor

^{۲۷} Move assignment operator

```

Vector(const Vector& a); // copy constructor: copy a into *this
Vector& operator=(const Vector& a); // copy assignment: copy a into *this
Vector(Vector&& a); // move constructor: move a into *this
Vector& operator=(Vector&& a); // move assignment: move a into *this
~Vector(); // destructor: destroy elements; release memory
// ...
};

```

عملگرهای گماردن باید هر منبعی که شیء مقصد مالکیت آنرا در اختیار دارد آزاد کند. عملیات جابجایی باید همه ی منابع را به شیء مقصد منتقل کند و اطمینان یابد که دیگر این منابع در اختیار شیء مبدا نیستند.

۲.۳. حذف کپی های افزونه

با داشتن این چهارچوب، کد **collect_lines** را مجدداً بررسی می کنیم. اول اینکه، می توان این کد را کمی ساده تر نوشت:

```

vector<string> collect_lines(istream& is) // collect unique lines from input
{
    unordered_set<string> s {from_range, istream_iterator<Line>{is}};
    return vector{from_range, s};
}
auto lines = collect_lines(cin);

```

istream_iterator<Line>{is}، به ما این امکان را می دهد که ورودی خوانده شده از مسیل **is** را به مثابه محدوده‌ای از خطوط در نظر بگیریم، تا اینکه مانند کد بخش ۱، عملیات ورودی را به شکل صریح و با زحمت بر روی مسیل ورودی اعمال کنیم. در این کد، و در خط آخر تابع، **vector** به خارج از **collect_lines()** بجای آنکه کپی شود، منتقل می شود. از حوالی سال ۱۹۸۳، کامپایلرها اینگونه عمل می کردند که مقدار بازگشت داده شده را (در اینجا **vector{from_ranges, s}**) در مقصد (در اینجا **lines**) می سازند. این کار به "ریزش کپی"^{۲۸} معروف است که از سال ۲۰۱۷ به بعد، این موضوع بخشی از استاندارد شد. هزینه‌ی ساختن با جابجایی، در بدترین حالت برای **vector**، کپی ۶ کلمه است، ۳ تا برای کپی نمایش رده‌ی **vector** و ۳ تا برای صفر کردن نمایش شیء اولی. این هزینه ی ناچیز حتی اگر بردار ما یک میلیون عضو داشته باشد صادق است. در مورد عمل گماردن با جابجایی، به هزینه فوق، سربار از بین بردن شیء مبدا اضافه خواهد شد. جابجایی، ما را از مدیریت حافظه‌ی گران و خطا خیز که صراحتاً باید با مدیریت نشانه‌روها صورت گیرد، رها می سازد.

پیش فرض، رشته نویسه از نوع **string** به صورت دنباله‌ای از نویسه های پشت سر هم که با فضای خالی از هم جدا شده‌اند، خوانده می‌شوند. اما من می‌خواهم یک خط را به همان معنای رایج خط بخوانم. در کتابخانه‌ی استاندارد گونه‌ای تحت عنوان **Line** نداریم. بنابراین با تکنیک زیر (با استفاده از برنامه سازی شیء گرا)، گونه‌ی **Line** را تعریف می کنیم:

```

struct Line : string { };
istream& operator>>(istream& is, Line& ln) { return getline(is, ln); }

```

کماکان می‌خواهیم به عنوان گونه‌ی بازگشت **collect_lines()**، از **vector<string>** استفاده کنیم. می‌شود این کار را کرد ولی با توجه به تعریف گونه‌ی **Line**، می‌توانیم گونه‌ی بازگشت را به **vector<Line>** تغییر دهیم.

بهرحال نویسه های یک **string** از **unordered_set** به درون **vector** همچنان کپی می شوند. این موضوع خود می-تواند هزینه بر باشد. در اساس، کامپایلر می تواند استنتاج کند که پس از پر کردن **vector**، از مسیل **s** استفاده‌ی دیگری نمی‌شود، پس می‌توان عضوهای آنرا را فقط جابجا کرد، اما درحال حاضر کامپایلرها آنقدر باهوش نیستند که بتوانند آنرا تشخیص دهند، بنابراین فعلا باید به صراحت از کامپایلر خواست که عناصر را جابجا کند:

```
vector<string> collect_lines(istream& is) // collect unique lines from input
{
    unordered_set<string> s{ from_range, istream_iterator<Line>{is} );
    return vector{from_range, std::move(s)}; // move elements, rather than copy
}
```

همچنان یک عمل کپی افزونه وجود دارد، نویسه ها از حافظه میانگیر ورودی در عضوهای مجموعه کپی می شوند. اگر این موضوع ایجاد مشکل می‌کند، می‌توان آنرا هم حذف کرد. برای نیل به این هدف به تکنیک‌های سطح پایین متداول، انتزاع-هایی ساده مانند **span** (بخش ۶.۲ به آن خواهیم پرداخت) و ارزیابی‌های زمان کامپایل (بخش ۵.۲ را ببینید) نیازمندیم. این کد خارج از حوصله این مقاله است و البته کمی شلوغ‌تر است اما، به‌مانند همیشه کدهای C++ با رابط‌های به خوبی مشخص شده، قابلیت تنظیم شدن دارند. همچنین خواهشمندم به یاد داشته باشید قبل از اندازه‌گیری و نیاز مبرم، به بهینه‌سازی کدی مبادرت نکنید. با C++ امروزین، رویکرد من به بهینه سازی، در وهله اول ساده سازی کد است. کدهای حاوی ساختمان داده های پیچیده و استفاده توام با زیرکی از نشانه‌روها نه تنها برنامه سازان، بلکه بهینه ساز کامپایلر را هم سردرگم می‌کند.

۳.۳. منابع و خطاها

یکی از اهداف کلیدی C++، ایمنی منابع است: هیچ منبعی نباید نشت کند. این بدان معناست که ما باید از نشت منابع حتی در شرایط وجود خطا هم جلوگیری کنیم. اصول بنیادین عبارتند از:

- جلوی نشت منابع را بگیرید.
 - منابع را در وضعیتی نامعتبر رها نکنید.
- بنابراین، هنگام بروز خطا، اگر خطایی تشخیص دادیم که نمی‌توان آنرا در جا مدیریت یا مهار کرد، قبل از خروج از تابع باید این کارها را انجام دهیم:
- هر شیء ای را که به آن دسترسی داشته ایم در وضعیت معتبر قرار دهیم.
 - هر شیء ای که تابع در برابر آن مسئولیت دارد باید آزاد شود.
 - در غیر اینصورت تابع بالاتر که در زنجیره فراخوانی قرار دارد به مسائل مرتبط با منبع رسیدگی خواهد کرد.
- این بدان معناست که "نشانه روهای خام" برای مهار منابع قابل اطمینان نیستند. فرض کنید گونه ای بنام **Gadget** داریم که دربرگیرنده منابعی چون حافظه، قفل و مهار فایل است:

```
void f(int n, int x)
{
    Gadget g{n};
    Gadget* pg = new Gadget{n}; // explicit new: don't!
    // ...
    if (x<100) throw std::runtime_error{"Weird!"}; // leaks *pg; but not g
    if (x<200) return; // leaks *pg; but not g
    // ...
}
```

استفاده‌ی صریح از عملگر **new**، برای قراردادن شیء **Gadget** در داخل حافظه آزاد و قرار دادن نشانی شیء در یک "نشانه روی خام" مشکل اصلی است، در حالیکه می‌توانستیم آنرا در یک مهار منبع که ویرانگری مناسب برای آن نوشته شده، انجام دهیم. همچنین، شیء‌های محلی ساده تر هستند و عموماً از عملگر **new** سریع تر هستند.

برای ساخت یک سیستم مطمئن، ما به سیاستی با مهارت و تفسیر شده برای مهار خطا نیازمندیم. بطور کلی، بهترین راهکار برای این موضوع، اینست که بین دو نوع خطای زیر تمایز قائل شویم:

- خطاهایی که می‌توانند به شکل محلی توسط فراخوان بلافصل مهار شوند.
- خطاهایی که فقط توسط کد فراخواننده در مرتبه‌ای بالاتر در زنجیره‌ی فراخوانی قابل مدیریت هستند. این خطاها باید از منظری فراخ‌تر و از نقطه نظر خواسته‌های سیستم مدیریت شوند:
- از کد خطا استفاده کرده و خطا را به شکل محلی آزمون کنید و در صورت امکان آنرا به شکل محلی مهار سازید
- از استثناء برای مدیریت شرایط نادر ("استثنایی")، استفاده کنید، شرایطی که نمی‌توان به شکل محلی از پس آنها برآمد:

▪ برای مهار خطا در ساختگرها، عملگرها و توابع دیگری که به آسانی نمی‌توانند کد خطا برگردانند (مثلاً: $\text{Matrix } x = y + z;$)

- برای حصول اطمینان از اینکه مشکل بوجود آمده به خطا پایان دهد، نه اینکه نتایج غلط ببار آورد.
- برای آنکه به شکل خودکار خطا را به سمت بالا در زنجیره فراخوانی منتشر کنیم.
- بدیل مدیریت استثناء، شیوه پرهزینه‌ی "جهنم کدهای خطا" است مهله‌ای که هر فراخوان، وقتی کدی را فراخوانی می‌کند باید وجود خطا را آزمون کند.

در برخی کاربردهای مهم، خاتمه دادن بدون شرط به برنامه‌گزینیهی مطلوب نیست. پس ما باید به‌یاد داشته باشیم تا تمامی کدهای خطا که از بخشهای مختلف برنامه بازگردانده می‌شود را، آزمون کرده و تمامی استثناء‌ها را در جایی از برنامه بقیه‌ییم^{۲۹} (مثلاً در **main()**) و برای آنها پاسخی مناسب آماده کنیم.

به رغم شگفتی بسیاری از برنامه‌سازان، استثناء و مدیریت آن، می‌تواند مقرون به صرفه‌تر و سریع‌تر از، استفاده سازگار از کدهای خطا باشد. این موضوع حتی برای سیستم‌های کوچک هم درست به نظر می‌رسد **[14;5]**.
کد زیر را در نظر بگیرید:

```
void fct(jthread& prod, jthread& cons, string name)
{
    ifstream in { name };
    if (!in) { /* ... */ } // possible failure expected
    // ...
    vector<double> constants {1, 1.618, 3.14, 2.99e8};
    vector<string> designers {"Strachey", "Richards", "Ritchie"};
    auto dmr = "Dennis M. " + designers[2];
    // ...
    pair<string, jthread&> pipeline[] { { "producer", prod },
                                       { "consumer", cons } };
    // ...
}
```

تصور کنید اگر مدیریت استثناء نبود برای این مثال کوچک (گرچه مصنوعی است اما غیر واقعی هم نیست) چقدر موارد مختلف را باید می آزمودیم؟ این مثال حاوی تخصیص حافظه، ساخت ساختارهای تو در تو، یک عملگر فزون بار (+) و در اختیار گرفتن یک منبع سیستمی (فایل **in**) می باشد.

شوربختانه استثناء، با اقبال عمومی مواجه نشده است و در همه آنجهایی که مناسب ترین است استفاده نمی شود. علاوه بر استفاده افراطی از نشانه‌روهای "خام"، مشکل دیگری هم وجود دارد و آن هم اینست که، بسیاری از توسعه‌دهندگان اصرار دارند که فقط از یک تکنیک برای گزارش دادن خطا استفاده کنند. یعنی، یا همگی می خواهند استثناءها را پرتاب کنند^{۳۰}، یا همه چیز را با برگرداندن کد خطا گزارش دهند. این شیوه با خواسته‌های جهان واقع تطابق ندارد.

۴. واحد مندی

پیش پردازنده ای^{۳۱} که **C++** از **C** به ارث برده است، همه جا استفاده می شود، ولی خود مانع عمده ای برای تولید و توسعه ابزار برنامه سازی است، کارایی کامپایلر را کاهش می دهد و همچنین منشاء خطاهای متنوع است. در گذشته از کلان دستورها^{۳۲} برای توصیف ثابت ها، توابع و گونه ها استفاده می شد. **C++** امروزین، کلان دستور را با گونه های مناسب تر، ثابت های محدود در یک حوزه، تابع های ارزیابی شونده در زمان کامپایل (بخش ۵.۳) و الگوها (بخش ۵.۱) جایگزین کرده است. اما، همچنان پیش پردازنده، برای بیان شکل سستی از واحد مندی^{۳۳} استفاده شده است. رابط کتابخانه ها و کدهای دیگر که جداگانه کامپایل شده اند - و قرار است در برنامه خود از آن استفاده کنیم - فایل هایی هستند حاوی کدهای **C++** که باید با کلان دستور **#include** ضمیمه گردند.

۴.۱. فایل های سرآیا

رهنمون **#include** متن یک فایل مانند فایل سرآیا^{۳۴} را، در داخل واحد ترجمه^{۳۵} کپی می کند. متأسفانه، این متضمن آن است که

```
#include "a.h"
#include "b.h"
```

به جهت معنایی با

```
#include "b.h"
#include "a.h"
```

به کل متفاوت باشد و این منشا اشکال های اساسی است.

رهنمون **#include**، تراگذرا است، یعنی اگر فایل **a.h** خود حاوی یک رهنمون مانند **#include "c.h"** باشد، محتوای **c.h** نیز بخشی از فایلی خواهد شد که از رهنمون **#include "a.h"** سود برده است. این نیز مشکلی دیگر است. از آنجا که فایلهای

^{۳۰} Throw

^{۳۱} Pre-processor

^{۳۲} Macro

^{۳۳} Modularity

^{۳۴} Header file

^{۳۵} Translation unit

سرآیا، اغلب بصورت چندتایی یا حتی چند صدتایی ضمیمه می‌شوند، همین موضوع خود به معنای ترجمه دوباره و چندباره‌ی یک فایل مبداء است که می‌تواند زمان کامپایل را بسیار طولانی سازد.

۲.۴. ماجول

این مشکلات که استفاده از فایل های سرآیا که تلاش می‌کنند تا به شکلی ساختگی واحد مندی را نمایش دهند، باعث آن است، قبل از تولد C++ شناخته شده بود. تعریف بدیلی برای این مکانیزم و باب کردن آن با وجود میلیاردها خط کد موجود کار آسانی نیست. اما حالا C++ ماژول را عرضه کرده که واحد مندی را در به شکلی مناسب عرضه می‌کند. نخست آنکه **import** کردن ماژول ها، مستقل از ترتیب آنها است، بنابراین:

```
import a;
import b;
```

به جهت معنایی با

```
import b;
import a;
```

کاملاً یکسان است.

نابستگی دوجانبه ی ماژول ها، به اصطلاح کد را "بهداشتی تر و پاکیزه تر" کرده و اشکال های مرتبط با وابستگی را غیرممکن می‌سازد.
در ادامه یک مثال ساده از تعریف ماژول را می بینیم:

```
export module map_printer; // we are defining a module
import iostream;           // we import modules needed for the implementation
import containers;
using namespace std;
export // this template is the only entity exported
void print_map(const Sequence auto& m) { // generic function; see section 5
    for (const auto& [key, val] : m) // access key and value pair from m
        cout << key << " -> " << val << '\n';
}
```

از آنجا که **import**، تراگذرا نیست، کاربر **map_printer**، به جزییات پیاده سازی لازم در **print_map**، دسترسی ندارد. به علاوه، ماژول صرفنظر از آنکه چند بار **import** شود فقط یکبار کامپایل می‌شود. این به معنای بهبود قابل توجه مدت زمان کامپایل است. بر اساس گزارش یک کاربر [3]:

```
#include <libgalil/DmcDevice.h> // 457440 lines after preprocessing
int main() // 151268 non-blank lines
{
    Libgalil::DmcDevice("192.168.55.10"); // 1546 milliseconds to compile
}
```

کامپایل این کد با بیش از نیم میلیون خط، فقط ۱.۵ ثانیه طول می‌کشد که به واقع سریع است. این در حالی است که کامپایلر در این زمان کوتاه واقعاً بیش از آنچه که باید انجام می‌دهد.

```
import libgalil; // 5 lines after preprocessing
int main() // 4 non-blank lines
{
    Libgalil::DmcDevice("192.168.55.10"); // 62 milliseconds to compile
}
```

این یعنی افزایش ۲۵ برابری سرعت کامپایل برنامه‌ها. به‌رحال بهبود ۷ تا ۱۰ برابری **import** در مقایسه با **#include** مشهود است. اگر این کتابخانه - libgalil- در ۲۵ فایل ضمیمه گردد، کامپایل آن بیش از نیم دقیقه طول می‌کشد درحالی‌که **import** فقط ۱.۵ ثانیه زمان ثبت کرده است. کل کتابخانه استاندارد در داخل یک ماژول قرار دارد. برنامه رایج "hello, world" را در نظر بگیرید [29]:

```
#include <iostream>
int main()
{
    std::cout << "Hello, World!\n";
}
```

بر روی کامپیوتر من، کامپایل این برنامه، ۸۷ صدم ثانیه طول می‌کشد. حال رهنمون **#include <iostream>** را با **import std;** جایگزین می‌کنیم. زمان کامپایل کد به ۸ صدم ثانیه افت می‌کند. این درحالیست که کل کتابخانه استاندارد با دستور **import**، در دسترس قرار گرفته است. این حجم اطلاعات { در مقایسه با **iostream** تنها } حداقل ۱۰ برابر بیشتر است. سازماندهی مجدد حجم زیادی کد { براساس پیش پردازنده و تبدیل آن به ماجول } نه ساده است و نه ارزان. **#include** در طول نیم قرن و از همان زمان تولد C در کدهای قدیمی نفوذ و رسوخ کرده است، به شکلی که برای بسیاری از برنامه‌سازان تصور C++ بدون **#include** دشوار است. به‌رحال در مورد ماجول باید گفت که، مزایای آن از نقطه نظر بهبود کیفیت کد، قابل توجه و از منظر زمان ساخت^{۳۶}، عظیم است.

۵. برنامه‌سازی هرویک

برنامه‌سازی هرویک^{۳۷} یا برنامه‌سازی عام، بنیان کلیدی C++ امروزی است. اهمیت این موضوع از قبل از تغییر نام C" با رده‌ها" به C++ فهم شده بود. اما فقط اخیراً و در ویرایش C++20، پشتیبانی زبان از این سبک برنامه‌سازی، به کمال مطلوب رسید [30].

برنامه‌سازی هرویک، به‌معنای برنامه‌سازی با گونه‌ها و توابعی که با گونه‌ها پارامتری شده‌اند، می‌باشد. این سبک برنامه‌سازی عرضه‌کننده ویژگیهای زیر است:

- کدها کوتاه‌تر و خواناتر می‌شوند.
- ایده‌ها در کد، با صراحت بیشتر بیان می‌شوند.

^{۳۶} Build time

^{۳۷} Generic Programming

- انتزاع‌های با سربار صفر^{۳۸} خواهیم داشت
- امنیت گونه بهبود می‌یابد

این به معنای کدهای کمتر، آزمون ساده‌تر کدها، نگهداری آسان‌تر و گاهی اوقات حتی کارایی بهبود یافته‌تر در مقایسه با سبک‌های دیگر برنامه‌سازی است.

الگوها، ابزار زبان C++ برای پشتیبانی از برنامه‌سازی عام هستند. الگوها در همه‌ی بخش‌های کتابخانه‌ی استاندارد حضور

دارند:

- ظرف‌ها و الگوریتم‌ها
 - همروندی: ریشه‌ها و قفل‌ها
 - مدیریت حافظه: تخصیص‌کننده‌های حافظه، مهار منابع (مانند **vector** و **list**)، **unique_ptr** ... و ورودی/خروجی
 - رشته‌نویسه‌ها و عبارتهای با قاعده
 - و موارد بسیار دیگر
- می‌توان کدهایی نوشت که با همه‌گونه‌های مناسب کار کنند. به عنوان مثال، در اینجا تابع **sort** را آورده‌ایم که با همه-ی گونه‌های قابل‌تصور که با تعریف "محدوده‌های مرتب‌شونده"^{۳۹} که در کتابخانه استاندارد مطابقت دارند، کار می‌کند:

```
void sort(Sortable_range auto& r);
vector<string> vs;
// ... fill vs ...
sort(vs);
array<int,128> ai;
// ... fill ai ...
sort(ai);
```

کامپایلر با اطلاعاتی که در اختیار دارد می‌تواند تشخیص دهد که آیا گونه‌های **vs** و **ai** نیازمندیهایی **Sortable_range** را پوشش می‌دهند یا خیر؛ و منظور از **Sortable_range** این است: محدوده‌ای با دسترسی تصادفی که مقادیر داده‌گونه‌ی آنرا بتوان مقایسه و جابجا کرد. این کمینه‌ترین خواسته برای عمل مرتب‌سازی است. اگر نشانوندها مناسب نباشند، کامپایلر در همان نقطه‌ی استفاده نادرست، پیام خطا خواهد داد. به عنوان مثال:

```
list<int> lsti;
// ... fill lsti ...
sort(lsti); // error: a list doesn't offer random access
```

بر اساس C++ استاندارد، یک **list**، محدوده‌ی مرتب‌شونده نیست، زیرا لیست‌های پیوندی دسترسی تصادفی را پشتیبانی نمی‌کنند.

۱.۵. الگو

پشتیبانی C++ از برنامه‌سازی هرویک، مبتنی بر سه هدف است:

- بینهایت عمومی و منعطف: "باید بتوان بیش از آنچه که قابل‌تصور است کار صورت داد"

^{۳۸} Zero-overhead abstractions

^{۳۹} Sortable range

- سربار صفر: مثلاً انتزاع‌هایی مانند **vector** یا **Matrix** در C++، باید با آرایه‌های C قابل رقابت باشند.
 - رابط‌های کاملاً مشخص: فزون‌باری بهبود یافته و پیام‌های خطاهای مناسب توسط کامپایلر عرضه شود.
- قالب سنتی تابع هرویک **sort** را در نظر بگیرید:

```
template<typename Random_access_iterator, typename Compare = std::less>
void sort(Random_access_iterator first, Random_access_iterator last,
         Compare pred)
{
    // ...
}
```

این سبک برنامه‌سازی برای دهه‌ها در C++ رایج بوده است. این تابع زوجی از تکرارگر^{۴۰} از گونه‌ای دلخواه را به عنوان ورودی می‌گیرد که دلالت بر محدوده‌ای از المانها در بازه‌ی - نیمه باز - (**first: last**) دارد و باید مرتب شود و همچنین تابع مقایسه‌کننده‌ی المانها که عملاً معیار مقایسه است. معیار مقایسه به شکل پیش فرض تابع "کمتر از" است یا به عبارت دیگر مرتب‌سازی از کوچکتر به بزرگتر یا همان صعودی است.

این تابع **sort** برای گستره‌ای از ظرفها، گستره‌ای از گونه‌های المانهای درون ظرفها و همچنین گستره‌ای از توابع معیار مرتب‌سازی قابل فراخوانی است. مثلاً:

```
int a[] = { 3,1,4,2,6,9,0,-1};
vector<string> v = { "CPL", "BCPL", "C", "C++" };
sort(begin(v), end(v)); // sort the vector
sort(begin(a), end(a)); // sort the array
sort(begin(v), begin(v) + size(v) / 2); // sort the first half of the vector
sort(begin(v), end(v), greater<string>{}); // sort in ascending order
// sort absolute values
sort(begin(a), end(a), [](int x, int y) { return abs(x) < abs(y); });
```

در کد فوق، این توضیحات لازم است:

- **greater<string>{}،** یک شیء تابع است که در کتابخانه‌ی استاندارد تعریف شده است. این شیء تابع، عملگر "بزرگتر از" را فراخوانی می‌کند { به عبارت دیگر می‌خواهیم المانهای **v** را به صورت از بزرگتر به کوچکتر یا نزولی مرتب کنیم }.
 - عبارت **[](int x, int y) { return abs(x) < abs(y); }**، در آخرین خط، عبارت **لاندای**^{۴۱} است که فراخوانی شده. این نمادگذاری دیگری است برای تولید شیء تابع که به C++ امروزی اضافه شده. چنین شیء تابع‌هایی، (شیء‌هایی با عملگر فراخوانی () و دارای قابلیت ذخیره کردن وضعیت) در توابع عام بسیار رایج هستند.
- همانطور که مشاهده می‌کنید، این سبک از برنامه‌سازی هرویک، بسیار عمومی، منعطف و مرسوم است اما کمی پرگو و همراه با دراز نویسی است. همچنین واریسی گونه با تاخیر (در زمان مورد سازی الگو^{۴۲}) انجام میشود که متضمن پیامهای خطای طولانی و گیج‌کننده است. پیش فرض این است که کامپایلر برای هر نشانوند گونه، که **sort** با آن فراخوانی می‌شود یک نسخه‌ی

^{۴۰} Iterator

^{۴۱} Lambda expression

^{۴۲} Template instantiation

جدا از پیاده سازی را ایجاد کرده، سپس آنرا بهینه می‌کند. اغلب کامپایلر بخشهای مشترک این کدها را ادغام می‌کند تا از آنچه که به فربه‌گی کد^{۴۳} معروف است جلوگیری شود.

به هر روی، این رابطها هدف بنیادین (اصل عمومیت) ما را نقض می‌کنند، چرا که رابطها باید به دقت مشخص شوند و دربرگیرنده‌ی اطلاعاتی باشند که اجازه‌ی واریسی گونه را (ترجیحاً در زمان کامپایل و اگر نشد در زمان اجرا) بدهند. فقط در این قرن نیل به تمامی اهداف سه‌گانه میسر شد و اکنون برنامه‌سازی عام در C++ به طور همزمان همه‌ی اصول را دربر می‌گیرد.

۲.۵. مفهوم

مفهوم^{۴۴} یا ادراک یک گزاره‌ی زمان کامپایل است. گزاره خود تابعی است که وقتی اجرا شود، یک مقدار بولی نتیجه خواهد داد. ادراک عمدتاً برای بیان نیازمندیهای پارامترهای یک الگو استفاده می‌شود. در اساس، روش اینگونه است که از گونه در مورد ویژگیهای مطلوبش پرسش شود. به طور مثال، "آیا تو محدوده‌ای هستی که بتوان آنرا مرتب کرد؟"، "آیا تو یک عددی؟ و یا "آیا عملیات مقایسه‌ای معمول را پشتیبانی می‌کنی؟"

اغلب یک ادراک، با مفاهیم دیگر ساخته می‌شود. مثلاً مفهوم **Sortable_range** که در الگوریتم **sort** که در بخش قبل ذکر آن رفت، استفاده شده

```
void sort(Sortable_range auto& r);
```

می‌تواند اینگونه ساخته شود:

```
template<typename R>
concept Sortable_range =
    random_access_range<R> // has begin()/end(), ++, [], +, ...
    && sortable<iterator_t<R>>; // can compare and swap elements
```

کد فوق بیان می‌کند که گونه‌ی **R** یک **Sortable_range** است اگر خود یک مفهوم **random_access_range** باشد و همچنین گونه تکرارگر آن با مفهوم **sortable** ادراک پذیر باشد. **random_access_range** و **sortable** مفهوم‌هایی هستند که قبلاً در کتابخانه استاندارد تعریف شده‌اند.

اما، این تعریف **Sortable_range** هنوز بطور کامل عمومیت ندارد تا به عنوان یک تابع در کتابخانه استفاده شود. مفهوم‌ها می‌توانند چندین نشانوند را بپذیرند، آنها صرفاً گونه‌ای از گونه‌ها نیستند. برای آنکه **sort** ای به واقع هرویک داشته باشیم، باید قادر باشیم معیار مقایسه را هم بیان کنیم. این کار به آسانی انجام می‌شود:

```
template<typename R, class Pred = ranges::less>
concept Sortable_range =
    random_access_range<R> // has begin()/end(), ++, [], +, ...
    && sortable<iterator_t<R>, Pred>; // compare elements using Pred
```

برای آنکه ساده‌ترین و رایج‌ترین عمل مرتب‌سازی را پیچیده نکنیم، تابع مقایسه را "کمتر از" (افزایشی) درنظر گرفته ایم که متداول است. با **Sortable_range** موجود، حال می‌توانیم نسخه محدوده‌ای **sort** را بنویسیم:

^{۴۳} Code bloat

^{۴۴} Concept

```
template<Sortable_range R, typename Pred = ranges::less>
void sort(R& r, Pred cmp = {});
```

و اینگونه از **sort** استفاده می کنیم:

```
vector v = { 1,5,2,8,-1 };
sort(v); // sort using <
sort(v, ranges::greater{}); // sort using >
```

می توان یک ادراک را مستقیماً، بر حسب عبارتهای زبانی (ونه بر اساس مفهوم های دیگر)، با آنچه که به "الگوی های استفاده" **[6]** معروف است، بیان کرد. به عنوان مثال:

```
template<typename T, typename U = T>
concept equality_comparable = requires(T a, U b) {
    {a==b} -> Boolean;
    {a!=b} -> Boolean;
    {b==a} -> Boolean;
    {b!=a} -> Boolean;
}
```

ساختارهای موجود در {...} باید معتبر بوده و گونه ای را برگردانند که با آنچه مفهوم پس از عملگر >- برمیگرداند تطابق داشته باشد و بتوان از آن سود برد. بنابراین، الگوهای استفاده ی فهرست شده، (مثلاً **a==b**) باید چیزی را برگردانند که به مثابه **bool** استفاده شود. توجه کنید که مفهوم **equality_comparable**، تبدیل های ضمنی و عملگر تساوی بر گونه های سازگار مختلف را مدیریت می کند (به عنوان مثال، گونه های اعداد صحیح و ممیز شناور). این موضوع برای مهار کدهای متداول C++ ضروری است. معمولاً، به مانند **sort**، واریسی گونه و اینکه گونه ها مطابقت داشته باشند به شکل ضمنی انجام می شود، اما می توان به شکل صریح با **static_assert** انجام داد:

```
static_assert(equality_comparable<int,double>); // succeeds
static_assert(equality_comparable<int>); // succeeds (U is defaulted to int)
static_assert(equality_comparable<int,string>); // fails
```

مفهوم **equality_comparable**، از قبل در کتابخانه ی استاندارد تعریف شده است و نیازی به تعریف مجدد آن نیست، اما چون مثال خوبی بود آنرا تعریف کرده و استفاده کردم. ما همیشه ادراک ها را داشته ایم. هر کتابخانه ی هرویک موفق به نوعی مفهوم ها را در خود دارد: در فکر یا اندیشه طراح، در مستندات و یا در توضیحات برنامه. آنها اغلب نمایش دهنده ی مفهوم های بنیادین حوزه ی آن کاربرد هستند. به عنوان مثال:

- گونه های توکار در زبان C/C++: گونه های حسابی و ممیز شناور **[35]**.
- کتابخانه استاندارد C++: تکرارگرها، دنباله ها و ظرفها
- ریاضیات: تک عملوندها^{۴۵}، گروه ها، حلقه ها و میدان ها
- گرافها: لبه ها و گره ها، گرافهای جهت دار ناچرخه ای، ...

C++20 ایده‌ی مفهوم را { به عنوان یک قابلیت نو بنیاد } عرضه نکرد، بلکه کاری که شد این بود که پشتیبانی مستقیم زبان به آن اضافه شد. همانطور که گفته شد، ادراک یک گزاره‌ی زمان کامپایل است. برنامه‌سازی با استفاده از ادراک‌ها آسانتر از برنامه‌سازی بدون آنها است. اما، به مانند هر ساختار جدیدی، باید یاد گرفت که چگونه آنرا به شکل موثر با امکانات دیگر زبان استفاده کرد.

۳.۵. ارزیابی زمان کامپایل

مفهوم یک نمونه از تابع‌های زمان کامپایل است. در C++ امروزین، هر تابع به کفایت ساده‌ای می‌تواند در زمان کامپایل ارزیابی شود:

- **constexpr**: می‌تواند در زمان کامپایل ارزیابی شود
 - **constexpr**: باید در زمان کامپایل ارزیابی شود
 - **concept**: در زمان کامپایل ارزیابی می‌شود و می‌تواند پذیرای گونه به عنوان نشانوند باشد.
- به عنوان مثال:

```
constexpr int isqrt(int n) // evaluate at compile time for constant arguments
{
    int i = 1;
    while (i*i<n)
        ++i;
    return i-1;
}
constexpr int s2 = isqrt(1234); // s2 is 35
```

ما می‌توانیم ارزیابی زمان کامپایل را هم بر گونه‌های توکار و هم تعریف شده توسط کاربر انجام دهیم. به طور مثال

```
constexpr auto yule = weekday(December/24/2024); // Tuesday
```

بدین سان ما محاسبات زمان کامپایل را با غنای گونه‌یی بالا بدست آورده‌ایم که ما را از نوشتن مهار خطا برای توابع بسیار زیادی بی‌نیاز خواهد کرد.

برای آنکه توابع **constexpr** و **concept** ها در زمان کامپایل قابل ارزیابی باشند، نمی‌توانند سه ویژگی زیر را دارا باشند:

- اثر جانبی داشته باشند
- به داده‌های غیر محلی تغییر پذیر دسترسی پیدا کنند
- رفتار تعریف نشده داشته باشند.

در هر حال، از این توابع به شکل دامنه دار و گسترده استفاده می‌شود، از جمله بخشهای زیادی از کتابخانه‌ی استاندارد. این توابع در حقیقت، پیاده‌سازی مفهوم تابع *خالص یا محض*^{۴۶} در C++ است، از سوی دیگر یک کامپایلر امروزین C++، در درون خود یک مفسر^{۴۷} نیز دارد و بنابراین می‌توان این توابع را در زمان کامپایل محاسبه کرد. همچنین ارزیابی در زمان کامپایل، یک مزیت برای کارآیی به حساب می‌آید.

^{۴۶} Pure function

^{۴۷} Interpreter

۶. رهنمون ها و الزام آوری

سبکهای امروزی، مزایای عمده ای را به بار می آورند. اما عاداتهای قدیمی جان سخت اند. مردم آشنا بودن را با ساده بودن اشتباه می گیرند. همچنین پرهیز از تکنیکهای نابهیینه دشوار است. اطلاعات مبهم، کهنه و مندرس بر روی وب می گردد و در مواد آموزشی دست به دست می شود. مدرن کردن کدهای موجود سخت و اغلب پرهزینه است. به علاوه، کدهای قدیمی رابطهای با سبک و سیاقی از رده خارج عرضه می کنند، به حدی که ادامه استفاده از سبکهای قدیمی را ترغیب می کند. ما باید به خود کمک کنیم تا به طراحی بهتر سوق پیدا کنیم.

پایداری/سازگاری، موضوعی اساسی است. به علاوه با وجود میلیاردها خط کد C++، فقط پذیرش تدریجی و آرام ویژگیها و تکنیکهای جدید زبان امکان پذیر است. پر واضح است که ما نمی توانیم زبان را عوض کنیم، ولی می توانیم شیوهی استفاده از آنرا تغییر دهیم. برنامه سازان، C++ ساده تری را مطالبه می کنند که بسیار منطقی است و همزمان جنبه های جدیدی را می خواهند و بر ادامه حیات و اجرای کدهای موجود پافشاری می کنند.

برای کمک به تمرکز توسعه دهندگان بر استفاده واقعی از C++ امروزی و پرهیز از "گوشه های تاریک و نخ نمای" زبان، مجموعه ای از رهنمون ها توسعه داده شده است. در اینجا، ما بر رهنمون های هسته زبان^{۴۸} تمرکز می کنیم که به عنوان هدفمندترین و هم راستا ترین با C++ ایده آل است [1].

مجموعه رهنمون ها باید عرضه کننده ی یک فلسفه منسجم از زبان برنامه سازی و مرتبط با شکل استفاده از آن باشد. این مجموعه باید بهترین تجربه های طراحی و برنامه نویسی در C++ امروزی را تدوین کند. اساسی ترین هدف من { مانند همیشه } استفاده ایمن گونه و ایمن منبع از C++ مبتنی بر استاندارد ایزو است. یعنی

- هر شیء ای باید منحصرأ براساس تعریفش استفاده شود
- هیچ منبعی نباید نشت شود.

نکات فوق دربرگیرنده ی ایمنی حافظه و موارد بسیار بیشتری نیز خواهند بود اگرچه به واقع چیز جدیدی نیست [24]. پر واضح است، نمی توان از این مجموعه رهنمون برای همه ی کاربردهای C++ استفاده کرد، اما اکنون ما تجربه ی زیادی به دست آورده ایم و با این تجربه می توان نشان داد که امکان استفاده از این مجموعه در کدهای مدرن وجود دارد، اگرچه هنوز الزام آور نیستند. مجموعه ی رهنمون ها نقاط قوت و ضعف دارد:

- درحال حاضر این رهنمون ها موجود است (به عنوان مثال رهنمون های هسته زبان که ذکر آن رفت [1]).
 - اجرای برخی از قاعده ها را می توان تحکم کرد و برخی را خیر.
 - درحال حاضر الزامی کردن اجرای رهنمون ها کامل نیست { و هنوز عملیاتی نشده است }
- ساخت برنامه ها بر پایه ی رهنمون ها نیازمند الزام است با توجه به موارد زیر:
- یک نمایه^{۴۹} که مجموعه ای منسجم و الزام آور از قاعده ها و رهنمون هاست [1, 30].
 - در گروه کاری ۵۰۲۱ و جاهای دیگر کار کند.
 - بجز در ویرایش های آزمایشی و ناقص، هنوز آماده نیست [16, 7, 15]

^{۴۸} C++ core guidelines

^{۴۹} Profiles

^{۵۰} WG21 - گروه کاری زبان برنامه سازی C++ که زیر نظر سازمان استانداردسازی جهانی یا ایزو، استانداردسازی این زبان را برعهده دارد.

هنگام اندیشیدن به C++، مهم است خاطرمأن باشد، که این تنها یک زبان - برنامه سازی- نیست، بلکه بخشی از یک زیست بوم^{۵۱} است، شامل پیاده سازی ها، کتابخانه ها، ابزارها، آموزش و بیشتر. به ویژه، توسعه دهندگان C++ بر تسهیلاتی تکیه دارند که بسی فراتر از آن چیزی است که در کتابهای C++ می توان یافت و در استاندارد ایزو تعریف شده است.

۱.۶. رهنمون ها

انتخاب ساده ی زیرمجموعه ای از C++ چندان کارساز نیست. به خصوص، ما همواره به جنبه های سطح پایین، دشوار و نیازمند مهارت، نزدیک به سخت افزار، خطاخیز و کارویژه نیازمندیم تا به کمک آنها قابلیت های سطح بالاتر را به شکلی کارآمد پیاده سازی کنیم. رهنمون های هسته زبان که ذکر آن رفت از یک راهکار، مشهور به زیرمجموعه ی/برمجموعه^{۵۲}، استفاده می کند [25]. رویه این است:

- **گسترش زبان:** ابتدا زبان را با چند انتزاع کتابخانه گسترش می دهیم، بدین شکل که بخش کوچکی از کتابخانه ی استاندارد را در نظر گرفته و به هسته ی زبان اضافه می کنیم بصورتی که بتوان از رهنمون به شکل مناسب و کارآمد استفاده کرد. به این کتابخانه ی منتخب، کتابخانه پشتیبانی از رهنمون^{۵۳} می گویند.
- **انتخاب زیرمجموعه ای از این ابر مجموعه:** در این مرحله استفاده از قابلیت های سطح پایین، ناکارآمد و خطاخیز، قدغن می شود.

آنچه که از این فرآیند گسترش-محدودسازی بدست می آید، "C++ ای قوی تر و بزرگتر" است. C++ ای ساده، ایمن، منعطف و سریع نه زیرمجموعه ای بی خاصیت و یا زبانی که صرفاً مبتنی بر بررسی های حجیم در زمان اجرا است. به علاوه ما زبانی با قابلیت های زیبا و/یا ناسازگار با یکدیگر ایجاد نمی کنیم. نتیجه C++ ای است که ۱۰۰٪ با استاندارد ایزو سازگار است. با این حال ویژگی های به هم ریخته، خطرناک و سطح پایین همچنان می توانند فعال شده و در صورت نیاز استفاده شوند. دامنه های کاربردی متفاوت، خواسته هایی متفاوت دارند بنابراین نیازمند مجموعه رهنمون های متفاوتی هم هستند. اما در آغاز تمرکز بر "رهنمون های هسته زبان" است. قواعدی که امیدواریم همه نهایتاً از آنها سود ببرند:

- از متغیرهای آغازین دهی نشده استفاده نکنید.
- محدوده یا نشانه روهای پوچ را تخریب نکنید.
- نشئت منابع ممنوع است.
- استفاده از نشانه روهای آونگان^{۵۴} ممنوع است.
- گونه ها را تخریب نکنید.
- نامعتبرسازی ممنوع است.

دو کتاب، C++ را بر اساس این رهنمون ها توصیف می کنند: اولی A Tour of C++ است که برای برنامه سازان مجرب نوشته شده است [30] و دیگری Programming: Principles and Practice using C++ که برای مبتدیان نوشته شده است [32]. دو کتاب دیگر نیز وجود دارد که جنبه هایی از رهنمون های هسته ی زبان را شرح می دهد [13,21].

^{۵۱} Ecosystem

^{۵۲} Subset-of-superset

^{۴۷} Guideline support library- GSL

^{۵۴} Dangling pointer

۲.۶. مثال: عملکرد اندیس گذاری را روی نشانه‌رو ها بکار نیندید!

یک نشانه رو، فاقد اطلاعات مرتبط و کافی برای واریسی محدوده است. اما، واریسی محدوده برای ایمنی حافظه و همچنین ایمنی گونه یک ضرورت به حساب می آید، زیرا نمی توان به برنامه اجازه داد که شیء ها را خارج از محدوده ای که به آن اشاره میشود بخواند و یا بازنویسی کند. در عوض، ما باید از انتزاع هایی استفاده کنیم که اطلاعات کافی برای واریسی محدوده داشته باشند، مانند آرایه ها، **vector** ها و **span** ها.

این سبک برنامه سازی را که رایج است در نظر بگیرید: یک نشانه رو به علاوه ی یک عدد صحیح که از قرار معلوم به تعداد المانهایی که به آنها اشاره می شود، دلالت دارد:

```
void f(int* p, int n)
{
    for (int i = 0; i<n; i++)
        do_something_with(p[i]);
}
int a[100];
// ...
f(a,100); // OK? (depends on the meaning of n in the called function)
f(a,1000); // likely disaster
```

این مثالی بسیار ساده از بکارگیری آرایه است به همراه اندازه ی آن. از آنجا که اندازه ی آرایه مهیا است، واریسی در نقطه ی فراخوانی گرچه با دشواری رو به روست ولی امکان پذیر است اما آنچه که در زنجیره ی طولانی تر فراخوانی ها ارسال می شود که بطور معمول یک زوج (نشانه رو، عدد صحیح) می باشد درست سنجی را دشوار یا حتی غیرممکن می کند. راه حل این است که اندازه ی آرایه را نه به شکل مجزا بلکه به نشانه رو متصل کنیم یا گره بزیم (مانند رده **Vector** که در بخش ۱.۳ توضیح داده شد). این همان کاری است که یک **span** انجام می دهد:

```
void f(span<int> a) // a span holds a pointer and the number of elements
                  // pointed to
{
    for (int& x: a) // now we can use a range-for
        do_something_with(x);
}
int a[100];
// ...
f(a); // the type and element count are deduced
f({a,1000}); // asking for trouble, but marked syntactically and easily
           // checkable
```

استفاده از **span** مصداقی از اصل "کارهای ساده را ساده کن" ^{۵۵} می باشد. کدی که از این اصل استفاده می کند ساده تر از

"سبک قدیمی" است: موجزتر، ایمن تر و اغلب سریع تر.

گونه ی **span**، ابتدا در کتابخانه پشتیبانی از رهنمون به مثابه یک گونه با واریسی محدوده اضافه شد. متأسفانه هنگام افزودن این گونه به کتابخانه ی استاندارد C++، ضمانت واریسی محدوده حذف شد. مسلماً، یک نمایه (بخش ۶.۴) می تواند این واریسی محدوده را الزام آور کند. همه ی پیاده سازی های مهم C++، روش هایی برای اطمینان از این الزام آوری دارند، مقاوم سازی کتابخانه

ی استاندارد در Clang [15]، ایمنی مکانی گوگل [7] و نمایه‌ی حالت اشکال زدایی و طول عمر از شرکت مایکروسافت [16] نمونه‌هایی از این نمایه‌ها هستند. شورشخانه هنوز برای این موضوع یک استاندارد و روشی حمل‌پذیر تعبیه نشده است.

۶.۳. مثال: از نشانه‌روهای بی اعتبار شده استفاده نکنید

برخی ظرفها، به خصوص **vector**، می‌توانند المانهای خود را بازجایی^{۵۶} کنند. اگر کسی خارج از ظرف، نشانه‌رویی را به یکی از المانها از آن خود کند و این نشانه‌رو را پس از بازجایی، استفاده کند، می‌تواند به فاجعه منجر شود. در نظر بگیرید:

```
void f(vector<int>& vi)
{
    vi.push_back(9); // may relocate vi's elements
}
void g()
{
    vector<int> vi { 1,2 };
    auto p = vi.begin(); // point to first element of vi
    f(vi);
    *p = 7; // error: p is invalid
}
```

با داشتن قاعده‌های مناسب برای استفاده از C++ (بخش ۶.۱)، آناکاو/ایستای^{۵۷} محلی می‌تواند از بی اعتبار شدن نشانه‌رو پیشگیری کند. در واقع، پیاده‌سازی‌های رهنمون‌های هسته زبان از سال ۲۰۱۹ به بعد این واریسی‌ها را انجام می‌دهند [16]. بطو کلی، ممانعت از بی اعتباری و بکارگیری نشانه‌روهای آونگان عملی ایستا یا زمان کامپایل است. به هیچ واریسی در زمان اجرا نیازی نیست بجز آنکه بررسی کنیم نشانه‌رو پوچ است (**nullptr**) یا خیر. واریسی طول عمر شیء ای که نشانه‌رو به آن نشانه می‌رود یا به معتبر بودن و یا **nullptr** بودن ختم می‌شود.

شرح پر جزئیات این آناکاو و شیوه عملکرد آن در این مقال نمی‌گنجد. برای کسب اطلاعات بیشتر به اینها مراجعه کنید [32,27,8]. به هر روی، نمای کلی این مدل عبارت است از:

قاعده‌های موجود را بر هر موجودیتی که مستقیماً به چیزی اشاره دارد می‌توان به کار بست. این موجودیت‌ها عبارتند از: نشانه‌روهای عادی، نشانه‌روهای مدیریت منابع، ارجاع‌ها و ظرفی از نشانه‌روها. **vector<int*> int& int***. یک نشانه‌روی هوشمند مانند **unique_ptr<int>**، یک شیء از گونه‌ی **jthread** که یک **int*** را در خود جای داده و یک عبارت لاندای که یک ارجاع به مقدار صحیح (**int&**) را تسخیر کرده، نمونه‌هایی از این موجودیت‌ها هستند.

- استفاده از نشانه‌رو را پس از **delete**، قدغن کرده و بر تکنیک اکتساب و آغازین دهی تکیه کنید.
- اجازه ندهید که نشانه‌رو از حوزه‌ای که در آن تعریف شده، بگریزد. منظور این است که یک تابع فقط وقتی می‌تواند یک نشانه‌رو را برگرداند که به چیزی { در حافظه‌ی { ایستا یا به چیزی در حافظه‌ی آزاد { یا حافظه‌ی شناور { اشاره کند و یا به عنوان نشانوند به تابع ارسال شده باشد.
- فرض کنیم یک تابع (مثلاً **vector::push_back()**)، که نشانوند غیر ثابت می‌گیرد، بی اعتبار گردد. در این حالت اگر نشانه‌رویی به یکی از المانها اشاره کند، باید این فراخوانی قدغن گردد. توابعی که فقط نشانوندهای ثابت را به عنوان ورودی می‌گیرند، اساساً نمی‌توانند بی اعتبار شوند، و همچنین به منظور جلوگیری از حجم زیاد خطای مثبت کاذب^{۵۸} و حفظ آناکاو محلی، می‌توانیم اعلان توابع را با **[profiles::non_invalidating]** حاشیه نویسی کنیم.

^{۵۶} Relocation

^{۵۷} Static analysis

^{۵۸} False positive

اعتبار این حاشیه نویسی، می تواند هنگام مشاهده تعریف تابع توسط کامپایلر سنجیده شود. بنابراین ایمن است و نه صرفاً توضیحی مانند "به من اعتماد کن".
طبیعتاً، جزئیات زیادی برای بحث و رسیدگی وجود دارد، اما برخی هنوز در حال گرفتن تاییدیه در نسخه های آزمایشی و برخی دیگر آماده برای فرستادن به پیاده سازی های نهایی هستند.

۴.۶. الزام آوری: نمایه ها

رهنمون ها خوب و مفید هستند، اما دنباله روی سازگار و پیوسته از آنها در یک پایگاه کد^{۵۹} بزرگ در اساس غیرممکن است. بنابراین الزام آور کردن رهنمون ها حیاتی است. در حال حاضر الزام آوری قواعد اعم از جلوگیری از آغازین دهی نکردن، خطای محدوده ها، ارجاع زدایی^{۶۰} `nullptr` و استفاده از نشانه روهای آونگان در دسترس بوده و مقرون به صرفه بودن آنها در پایگاه کدهای بزرگ به نمایش درآمده است [16, 7, 15].

اما، قواعد کلیدی و بنیادین باید استاندارد باشند - یعنی باید بخشی از تعریف C++ بشوند - تا بتوان با شیوه ای استاندارد در نوشتن کدها آنها را بکار گرفت، بدین سان /ندر عمل پذیری^{۶۱} بین کد توسعه یافته توسط سازمانهای مختلف، سکوهای مختلف و کد استفاده شده در حوزه ی آموزش به وجود خواهد آمد.

به مجموعه ای منسجم و الزام آور از رهنمون ها که تامین کننده ی تضمین هایی هستند، "نمایه" گفته می شود. در حال حاضر دسته رهنمون های زیر برای استاندارد کردن برنامه ریزی شده اند و آماده قرار گرفتن در نمایه هستند (پر واضح است که این موارد سالها در نمایه های رهنمون های هسته ی زبان استفاده می شده اند) [10, 1]:

- **گونه** - هر شی ای باید آغازین دهی شود، هر گونه قالب دهی^{۶۲} ممنوع است. استفاده از - ساختمان داده ی - /اجتماع^{۶۳} ممنوع است.
 - **طول عمر** - دسترسی به واسطه ی نشانه رو های آونگان ممنوع است. در ارجاع زدایی نشانه روهها، پوچ بودن نشانه رو واریسی شود، استفاده صریح از عملگرهای `new/delete` ممنوع است.
 - **کران ها** - محدوده ی همه ی اندیس گذاری ها واریسی شود، عملیات حسابی بر روی نشانه روهها ممنوع است.
 - **عملیات حسابی** - سرریز و ته ریز ممنوع است. تبدیل گونه از علامت دار به بی علامت و برعکس که باعث تغییر مقدار یک متغیر شود ممنوع است.
- این موارد را می توان به مثابه "هسته ای درون هسته" توصیف کرد که شرح آن در بخش ۶.۱ رفت. با فرض وجود زمان و آزمون گری های کافی، نمایه های بیشتر هم در آینده اضافه خواهد شد [33]. مثلاً:
- **الگوریتم ها** - در تمامی محدوده ها، هیچ ارجاع زدایی از تکرارگر `end()` نباید صورت گیرد.
 - **همروندی** - بن بست ها و مسابقه ی داده ها باید حذف شوند (اگرچه کار سختی است)
 - **تکنیک اکتساب و آغازین دهی** - هر منبعی باید در مالکیت یک مهار باشد. (نه فقط حافظه که با `new/delete` مدیریت می شود، بلکه هر منبعی!)

همه ی نمایه ها، استاندارد ایزو نیستند. انتظار داریم که در گذر زمان، نمایه هایی برای حوزه های کاربردی خاص مانند پویا نمایی، نرم افزارهای هوا و فضا و محاسبات علمی مطرح شود. این موضوع، این حقیقت را بارتاب می دهد که، "ایمنی" امری تکینه نیست. دامنه ها و حوزه های مختلف به وادار سازی مجموعه های مختلف از قاعده ها نیاز دارند تا به خواسته هایشان برسند.

^{۵۹} Code base

^{۶۰} Dereference

^{۶۱} Interoperability

^{۶۲} Type cast

^{۶۳} Union

در اساس وادارسازی، ایستا یا رمان کامپایل است اما بهر حال چند واریسی مهم باید در زمان اجرا انجام شود، (مثلاً اندیس گذاری و ارجاع زدایی نشانه روها).
یک نمایه باید به صراحت برای یک واحد ترجمه درخواست گردد. به عنوان مثال:

```
[[profiles::enforce(type)]] // no casts or uninitialized objects in this TU
```

در صورت لزوم، می توان یک نمایه را برای یک حکم (یا حکم های مرکب) منکوب کرد. به طور مثال:

```
// no dangling pointer checks for the next statement  
[[profiles::suppress(lifetime)]]  
this->succ = this->succ->succ;
```

غیر فعال سازی درست سنجی ضمانت ها یا همان نمایه، اساساً به دلیل پیاده سازی انتزاع هایی است که اگر چه این ضمانت ها را تامین می کنند (مانند `vector`، `span` و `string_view`)، واریسی محدوده انجام می دهند، اما همزمان باید مستقیماً به سخت افزار هم دسترسی داشته باشند. از آنجا که C++ نیاز دارد تا با سخت افزار مستقیماً کار کند، نمی توان پیاده سازی انتزاع های بنیادین را به زبانهای دیگر "برون سپاری" کرد. به علاوه - به دلیل طیف گسترده ی کاربردها و پیاده سازی های چندگانه و مستقل - نمی توان پیاده سازی انتزاع های بنیادین را به سادگی به کامپایلر سپرد. برای لحظه ای تصور کنید که همه انتزاع های درگیر با ساختمان داده های پیوندی توسط کامپایلر C++ پیاده سازی شوند!

پر واضح است که، بی اثر سازی ها باید کمینه باشد. اما همانطور که نشان داده شد، محلی هستند و به راحتی می توان آنها را جستجو کرد. به علاوه، منکوب کردن بر ضمانتی ویژه تمرکز دارد و به معنای هیچ یا همه چیز نیست. بیشتر خواسته های ناظر بر ایمنی صرفاً خصوصیات زبان برنامه سازی نیستند، بلکه نیازمندیهای سیستمی درگیر با ابزار و تکنیک ها هستند و به نوعی ورای آن چیزی هستند که بتواند توسط زبان برنامه سازی عرضه شود. هدف گذاری نمایه ها، تامین ضمانتهای پالوده ی زبانی است، اما آنها نمی توانند جایگزینی برای بسیاری از ابزارها و تکنیک ها باشند که امروزه توسط بسیاری برای توسعه ی سامانه های اطمینان پذیر به کار برده می شوند. داشتن چندین نمایه واقعی را بازتاب می دهد: یک رویکرد، مناسب برای همه و هر هدفی نیست.^{۶۴}

۷. آینده

من چندان تمایلی ندارم که آینده را پیش بینی کنم، بخشی بخاطر آنکه اساساً پرمخاطره است، و بخشی بخاطر آن است که تعریف { و تکامل C++ } توسط یک کمیته استاندارد سازی بزرگ کنترل می شود که عملکرد آن مبتنی بر اجماع است. آخرین باری که بررسی کردم، کمیته استاندارد سازی ۵۲۷ عضو داشت. این بیانگر اشتیاق، علائق گسترده و همزمان عرضه کننده کار ویژه های متنوع است ولی لزوماً این رویکرد، برای طراحی یک زبان برنامه سازی کمال مطلوب نیست.
در کنار موضوعات دیگر، موارد زیر در حال پیشرفت است:

- یک مدل عمومی برای رایانش ناهمگام^{۶۵} [17]
- بازتاب ایستا^{۶۶} [2]

^{۶۴} No one size fits for all

^{۶۵} Asynchronous computing

^{۶۶} Static reflection

- مدل های تک دستورالعمل-چند داده ای^{۶۷} [18]
- سیستم قراردادهای^{۶۸} [12]
- تطبیق الگو^{۶۹} به سبک برنامه سازی تابعی^{۷۰} [9,19]
- یک سیستم واحد عمومی (مانند سیستم واحد بین المللی^{۷۱}) [20]

نسخه ی آزمایشی کلیه ی این ابزارها در دسترس است.

یک نگرانی جدی این است که چگونه این ایده های متنوع، را در قالب یک کل منسجم، یکپارچه سازیم. طراحی زبان، درگیر شدن با تصمیم سازی در فضایی است که لزوماً همه عوامل مرتبط با هم از قبل شناخته شده نیستند و همچنین نتایج پذیرفته شده برای دهه ها امکان تغییر چشمگیر ندارند. طراحی و تکامل زبان، با تولید محصولات نرم افزاری و اکثر پیشه های علوم کامپیوتر متفاوت است. شاهد این مدعا، تلاش برای طراحی زبانهای زیادی است که در طول دهه ها انجام شد و با شکست مواجه شدند و این خود نمایانگر جدی بودن این حوزه است.

۸. خلاصه

C++ به شکلی طراحی شد که تکامل یابد. وقتی { دهه ها پیش } کار را آغاز کردم، نه تنها منابع کافی برای طراحی و پیاده سازی زبان ایده آل خود در اختیار نداشتم، بلکه متوجه شدم که لازم است از کاربری آن بازخورد بگیرم تا کمال مطلوبی را که به دنبال آن هستم به واقعیت کاربردی تبدیل کنم. تکامل C++ به شکلی بوده که به اصول بنیادین خود وفادار بماند [24]. C++ امروزین (C++23) تقریبی بسیار بهتر نسبت به نسخه های قبلی است، از جمله پشتیبانی بهتر از کیفیت کد، امنیت گونه، قدرت تبیین ساختارها، کارآمدی و نهایتاً تخمینی بسیار بهتر برای گستره ی وسیعی از کاربردها.

اما، این رویکرد تکاملی باعث ایجاد مشکلاتی جدی شد. بسیاری از برنامه سازان در منظره منسوخ از C++ گیر کرده اند. امروزه، همچنان و ظاهراً قرار است به شکلی پایان ناپذیر از یک زبان خیالی بنام C/C++ نام برده شود. این موضوع متضمن آن است که C++ را صرفاً به عنوان یک گسترش کهاد از C در نظر بگیریم که در برگیرنده همه ی آن جنبه های نامناسب C و بد بکار بستن عجیب و غریب ویژگی های پیچیده ی C++ می باشد. در برخی دیگر از منابع از C++ به عنوان یک تلاش ناموفق از طراحی زبان جاوا یاد می شود. همچنین پشتیبانی از ابزارهایی مانند مدیریت تولید بسته ی نرم افزاری و سیستم های تولید کد، عقب افتاده است که دلیل آن تمرکز جامعه ی زیستی C++، بر استفاده از سبکهای قدیمی تر است.

مدل زبان C++، را می توان اینگونه خلاصه کرد:

- سیستم گونه ی ایستا
 - پشتیبانی یکسان از داده گونه های توکار و تعریف شده توسط کاربر
 - پشتیبانی از دو رویکرد سمانتیک مقداری و سمانتیک ارجاعی
- مدیریت منابع عام و سیستماتیک (بر اساس تکنیک در اختیار گرفتن منبع به معنای آغازین دهی است)
- برنامه سازی شیء گرا و در عین حال کارآمد
- برنامه سازی هرویک و در عین حال منعطف و کارآمد
- برنامه سازی زمان کامپایل

^{۶۷} Single-Instruction Multiple-Data- SIMD

^{۶۸} Contract system

^{۶۹} Pattern matching

^{۷۰} Functional programming

^{۷۱} System International of Units- SI

- استفاده مستقیم از منابع سخت افزار و سیستم عامل
- پشتیبانی از همروندی با استفاده از کتابخانه ها
- حذف پیش پردازشگر C

زبان برنامه سازی C++ و کتابخانه‌ی استاندارد بیان منسجمی از این مدل است و بخشی حیاتی از زیست بومی است که در تولید و توسعه C++ استفاده می شود. { در پایان دوباره متذکر می شویم } ارزش یک زبان برنامه‌سازی به کیفیت برنامه هایی است که با آن نوشته می‌شود.

۹. مرجع ها

- [1] B. Stroustrup and H. Sutter (editors): C++ Core Guidelines.
<https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md>
- [2] W. Childers et al: Reflection for C++26. WG21 P2996R6. 2024. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2996r6.html>.
- [3] D. Engert: A (Short) Tour of C++ Modules. CppCon 2021,
<https://www.youtube.com/watch?v=nP8QcvPpGeM&t=7s>
- [4] D. Engert: Contemporary C++ in Action. CppCon 2022.
<https://www.youtube.com/watch?v=yUIFdL3D0Vv>.
- [5] GCC exception performance.
<https://databasearchitects.blogspot.com/2024/12/c-exception-performance-three-years.html>.
- [6] G. Dos Reis and B. Stroustrup: Specifying C++ Concepts. POPL06. 2006.
<https://www.stroustrup.com/popl06.pdf>
- [7] A. Rebert et al: Retrofitting Spatial safety to hundreds of millions of lines of C++. 2024.
<https://security.googleblog.com/2024/11/retrofitting-spatial-safety-to-hundreds.html>
- [8] H. Sutter: Lifetime safety: Preventing common dangling. WG21 P1179. 2019-11-22.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1179r1.pdf>
- [9] H. Sutter: Pattern matching using is and as. WG P392R3. 2024.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2392r0.pdf>
- [10] H. Sutter: Core safety profiles for C++26. WG21 D3081R1. 2025.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3081r1.pdf>
- [12] J. Berne, T. Doumler, and A. Krzemieński: Contracts for C++. P2900R9. 2024.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2900r12.pdf>
- [13] J. Davidson and K. Gregory. Beautiful C++: 30 Core Guidelines for Writing Clean, Safe, and Fast Code. 2021. ISBN 978-0137647842.
- [14] K. Estell: C++ Exceptions for Smaller Firmware. CppCon (2024),
<https://www.youtube.com/watch?v=bY2FlayomIE&t=1878s>.
- [15] K. Varlamov and L. Dionne: Standard library hardening. WG21 P3471R0. 2024. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3471r0.html>
- [16] K. Reed: Lifetime Profile Update in Visual Studio 2019 Preview 2. 2019.
<https://devblogs.microsoft.com/cppblog/lifetime-profile-update-in-visual-studio-2019-preview-2/>
- [17] L. Baker et al: A plan for std::execution for C++26. WG21 P3109R0. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3109r0.html>.
- [18] M. Kretz: std::simd — data-parallel types. WG21 P21928R9. 2024. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p1928r9.pdf>
- [19] M. Park: Pattern Matching: match Expression. WG21 P2688R2. 2024. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2688r2.html>
- [20] M. Pusz et al: Quantities and units library. WG21 P3045R2. 2024. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3045r2.html>
- [21] R. Grimm: C++ Core Guidelines Explained. Addison-Wesley. 2022. ISBN 978-0136875673.

- [22] B. Stroustrup: Classes: An Abstract Data Type Facility for the C Language. SIGPLAN Notices, January 1982. <https://dl.acm.org/doi/10.1145/947886.947893>.
- [23] B. Stroustrup: A History of C++: 1979-1991. ACM SIGPLAN. March 1993. <https://www.stroustrup.com/hopl2.pdf>
- [24] B. Stroustrup: The Design and Evolution of C++. Addison Wesley, ISBN 0-201-54330-3. 1994.
- [25] B. Stroustrup: A rationale for semantically enhanced library languages. LCSD05. October 2005. <https://www.stroustrup.com/SELLrationale.pdf>
- [26] B. Stroustrup: Evolving a language in and for the real world: C++ 1991-2006. ACM SIGPLAN. June 2007. <https://dl.acm.org/doi/10.1145/1238844.1238848>.
- [27] B. Stroustrup, H. Sutter, and G. Dos Reis: A brief introduction to C++'s model for type- and resource-safety. Isocpp.org. October 2015. Revised December 2015. <https://www.stroustrup.com/resource-model.pdf>
- [28] B. Stroustrup: Thriving in a Crowded and Changing World: C++ 2006–2020. ACM SIGPLAN. June 2020. <https://dl.acm.org/doi/10.1145/3386320>.
- [29] B. Stroustrup: Minimal module support for the standard library. WG21 P2412r0. 2021. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2412r0.pdf>.
- [30] B. Stroustrup: A Tour of C++ (3rd Edition). Addison-Wesley. 2022. ISBN 978-0-13-681648-5.
- [31] B. Stroustrup and G. Dos Reis: Design Alternatives for Type-and-Resource Safe C++. WG21 P2687R0. 2022. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2687r0.pdf>.
- [32] B. Stroustrup: Programming: Principle and Practice using C++. Addison-Wesley. 2024. ISBN 978-0-13-830868-1.
- [33] B. Stroustrup: A framework for Profiles development. WG21 P3274R0. 2024. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3274r0.pdf>
- [34] B. Stroustrup: Profile invalidation – eliminating dangling pointers. WG21 P3346R0. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3446r0.pdf>
- [35] B.W. Kernighan and D.M. Ritchie: The C Programming Language. Prentice-Hall. 1978. ISBN 01-13-110163-3.

قدردانی

مایل هستیم از خوانندگان نسخه‌ی پیشنویس این مقاله تشکر کنیم: ژوزف کانرو، جیمز کیوسیک، ونسان لکستری، کریستوف میروالد، نیکولاس استراستروپ، آندریاس ویز، جی سی ون وینکل، مایکل ونگ، و همچنین از کلیه مشارکت کنندگان در کارگروه رهنمون‌های هسته C++ و عرضه کنندگان پیشنهادهای درحال تکامل برای نمایه‌های استاندارد ایزو تشکر می‌کنیم.